



7.1. Operations on Arrays

Previous Section:

7. Arrays

Contents:

1. Traversal

2. Insertion and Deletion

2.1. Insertion at the Beginning

2.2. Insertion at the End

2.3. Insertion at a Certain Position

2.4. Deletion at the Beginning

2.5. Deletion at the End

2.6. Deletion at a Certain Position

3. Indexing and Slicing

3.1. Indexing

3.2. Slicing

4. Complete Array Class

Summary

References

Now that we understand what arrays are and why they are important, it is time to actually work with them.

In practice, arrays are rarely used as isolated pieces of data. Instead, they are usually encapsulated inside classes, objects, or larger systems that manage how data is stored and manipulated. For this reason, throughout this chapter—and in

most of the data structures that follow—we will implement our structures using **Object-Oriented Programming (OOP)**.

Think of an array as a container. Creating the array is only the beginning. The real power comes from the operations we perform on it:

- Traversing elements
- Inserting new data
- Deleting existing data
- Searching for values
- Updating information

These operations form the foundation of almost every data structure in computer science. Whether we are working with linked lists, stacks, queues, trees, or graphs, we will repeatedly encounter the same fundamental actions.

To demonstrate these operations, we will create our own `Array` class that simulates a fixed-size array. The class stores:

- `maximum` → the maximum capacity
- `n` → the current number of elements
- `arr` → the actual storage area

```
class Array:
    def __init__(self, maximum):
        self.maximum = maximum
        self.n = 0
        self.arr = [0] * maximum
```

With this structure in place, we can begin performing operations on the array.

1. Traversal

Traversal is one of the most fundamental operations in any data structure. Before we can manipulate data, we often need to inspect it. Traversal simply means visiting every element currently stored inside the structure.

For arrays, traversal is straightforward because the elements are stored sequentially in memory. We simply start from the first element and continue until we reach the last occupied position.

And eventually every element has been visited. The time complexity of traversal is: $O(n)$, because every element must be visited exactly once.

Implementation:

```
class Array:
    def __init__(self, maximum):
        self.maximum = maximum
        self.n = 0
        self.arr = [0] * maximum

    def traverse(self):
        print(self.arr[:self.n])
```

```
a = Array(10)

a.arr[0] = 10
a.arr[1] = 20
a.arr[2] = 30
a.n = 3

a.traverse()
```

[10, 20, 30]

Notice that we only display elements up to `n`, which represents the current length of the array.

2. Insertion and Deletion

Insertion and deletion are arguably the two most important operations in every data structure.

Insertion allows us to add new data into a structure, while deletion allows us to remove data that is no longer needed.

These operations appear everywhere:

- Operating systems managing processes
- Databases storing records

- Social media applications handling posts
- Games managing objects and inventories
- Machine learning systems processing datasets

Understanding insertion and deletion is therefore essential before moving on to more advanced structures.

For our fixed-size array, there is one important rule:

The number of stored elements must never exceed the maximum capacity of the array.

If the array is already full and we try to insert another element, an **Overflow** condition occurs.

Similarly, if we attempt to remove an element from an empty array, an **Underflow** condition occurs.

2.1. Insertion at the Beginning

Insertion at the beginning is one of the most expensive insertion operations. Why? Because the first position is already occupied. To create space, every existing element must be shifted one position to the right.

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 81 | 67 | 20 | 56 |      |      |      |      |      |      |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Insert 90 at the beginning:

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 90 | 81 | 67 | 20 | 56 |      |      |      |      |      |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Steps:

1. Shift all elements right.
2. Place the new value at index 0.
3. Increase the current size.

Time Complexity: $O(n)$ because every element may need to be moved.

Implementation:

```
def insert_begin(self, item):
    if self.n < self.maximum:

        for i in range(self.n, 0, -1):
            self.arr[i] = self.arr[i - 1]

        self.arr[0] = item
        self.n += 1

    else:
        print("Overflow")
```

2.2. Insertion at the End

Insertion at the end is the simplest and fastest insertion operation. Since the next available position is already known (`n`), we simply place the new value there.

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 81 | 67 | 20 | 56 |   |   |   |   |   |   |   |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Insert 90 at the end:

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 81 | 67 | 20 | 56 | 90 |   |   |   |   |   |   |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

No shifting is required. Time Complexity: $O(1)$

Implementation:

```
def insert_end(self, item):
    if self.n < self.maximum:

        self.arr[self.n] = item
        self.n += 1
```

```

else:
    print("Overflow")

```

2.3. Insertion at a Certain Position

Sometimes we need to insert an element somewhere in the middle of the array. To do this:

1. Shift all elements after the position one step right.
2. Insert the new value.
3. Increase the size.

```

+---+---+---+---+---+---+---+---+---+
| 81 | 67 | 20 | 56 |   |   |   |   |   |
+---+---+---+---+---+---+---+---+

```

Insert 90 at index 2:

```

+---+---+---+---+---+---+---+---+---+
| 81 | 67 | 90 | 20 | 56 |   |   |   |   |
+---+---+---+---+---+---+---+---+

```

Time Complexity: $O(n)$

```

def insert_position(self, item, pos):
    if self.n < self.maximum:

        if 0 <= pos <= self.n:
            for i in range(self.n, pos, -1):
                self.arr[i] = self.arr[i - 1]

            self.arr[pos] = item
            self.n += 1

        else:
            print("Out of bound")

```

```

else:
    print("Overflow")

```

2.4. Deletion at the Beginning

Deleting the first element requires all remaining elements to shift one position to the left.

```

+---+---+---+---+---+---+---+---+---+---+
| 81 | 67 | 20 | 56 |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+---+---+

Delete 81:

+---+---+---+---+---+---+---+---+---+---+
| 67 | 20 | 56 |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+---+---+---+

```

Steps:

1. Shift all elements left.
2. Clear the final position.
3. Decrease the size.

Time Complexity: $O(n)$

Implementation:

```

def delete_begin(self):
    if self.n > 0:

        for i in range(self.n - 1):
            self.arr[i] = self.arr[i + 1]

        self.arr[self.n - 1] = 0
        self.n -= 1

    else:
        print("Underflow")

```

2.5. Deletion at the End

Deleting the final element is very efficient. We simply reduce the current size of the array by one.

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 81 | 67 | 20 | 56 |      |      |      |      |      |      |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Delete 56:

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 81 | 67 | 20 |      |      |      |      |      |      |      |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Time Complexity: $O(1)$

Implementation:

```
def delete_end(self):
    if self.n > 0:
        self.n -= 1
        self.arr[self.n] = 0
    else:
        print("Underflow")
```

2.6. Deletion at a Certain Position

Deleting an element in the middle requires shifting subsequent elements left.

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 81 | 67 | 20 | 56 |      |      |      |      |      |      |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Remove 56:

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```


	81		67		20														
+	-	-	-	+	-	-	-	+	-	-	-	+	-	-	-	+	-	-	+

Steps:

1. Shift all following elements left.
2. Clear the final position.
3. Reduce the size.

Time Complexity: $O(n)$

Implementation:

```
def delete_position(self, pos):
    if 0 <= pos < self.n:
        for i in range(pos, self.n - 1):
            self.arr[i] = self.arr[i + 1]

        self.arr[self.n - 1] = 0
        self.n -= 1

    else:
        print("Underflow")
```

At this point, we can already perform the most important modifications on an array: traversing existing elements, inserting new data, and deleting unwanted values. These operations form the foundation of array manipulation and will continue to appear throughout the study of more advanced data structures.

3. Indexing and Slicing

So far, we have learned how to traverse, insert, and delete elements from an array. However, there is another important question:

| How do we access specific data inside the array?

In everyday programming, we rarely want to inspect every element one by one. Instead, we often want to locate a particular value or extract only a portion of the array. This is where **indexing** and **slicing** become useful.

Indexing allows us to identify the position of an element inside the array, while slicing allows us to retrieve a subset of elements between two positions.

These operations are commonly used in data analysis, machine learning, databases, and virtually every application that works with collections of data.

3.1. Indexing

Indexing refers to locating the position of an element within the array.

For example, consider the following array:

```
Index      0      1      2      3      4
+-----+-----+-----+-----+
Array      | 10 | 20 | 30 | 40 | 50 |
+-----+-----+-----+-----+
```

Suppose we want to find the position of the value **30**.

The simplest approach is to start from the first element and compare each value until the target is found.

```
+-----+-----+-----+-----+
| 10 | 20 | 30 | 40 | 50 |
+-----+-----+-----+-----+
  ↑
Not 30

+-----+-----+-----+-----+
| 10 | 20 | 30 | 40 | 50 |
+-----+-----+-----+-----+
      ↑
      Not 30

+-----+-----+-----+-----+
| 10 | 20 | 30 | 40 | 50 |
+-----+-----+-----+-----+
                ↑
                Found!!
```

If the element exists, we return its index. If the element does not exist, we return `-1`.

Time Complexity: $O(n)$ because we may need to examine every element in the worst case.

Implementation:

```
def find_element(self, item):  
  
    for i in range(self.n):  
        if self.arr[i] == item:  
            return i  
  
    return -1
```

```
a = Array(10)  
  
a.insert_end(10)  
a.insert_end(20)  
a.insert_end(30)  
a.insert_end(40)  
  
print(a.find_element(30))
```

2 → Because `30` is located at index `2`.

3.2. Slicing

Sometimes we do not need the entire array. Instead, we may only need a portion of it. This operation is called **slicing**.

A slice extracts elements between two indexes:

$$\text{start} \leq \text{index} < \text{end}$$

Notice that the ending index is **excluded**. For example:

Slice from Index 1 to 4

```
+-----+-----+-----+-----+
| 10 | 20 | 30 | 40 | 50 |
+-----+-----+-----+-----+
      ↑       ↑
```

The value at index `4` is not included.

Slicing is frequently used when selecting a range of data, extracting records from datasets, working with matrices, preparing training and testing data in Machine Learning

Implementation:

```
def slice_element(self, start, end):

    if 0 <= start <= end <= self.n:
        return self.arr[start:end]

    return []
```

```
a = Array(10)

a.insert_end(10)
a.insert_end(20)
a.insert_end(30)
a.insert_end(40)
a.insert_end(50)

print(a.slice_element(1, 4))
```

[20, 30, 40]

Notice that the element at index `4` (`50`) is not included because slicing stops at `end - 1`.

4. Complete Array Class

At this point, our array supports the most important operations:

- Traversal
- Insertion
- Deletion
- Indexing
- Slicing

Combining everything together gives us a simple but fully functional array implementation.

Array.py

This implementation may appear simple, but it already demonstrates the core concepts behind how arrays operate internally. More advanced data structures such as stacks, queues, heaps, and dynamic arrays are built upon these same fundamental operations.

Summary

In this section, we implemented a custom array class and explored the most important operations that can be performed on an array. We learned how to traverse elements, insert new values at different positions, remove existing elements, locate data using indexing, and extract subsets of data through slicing.

Along the way, we also examined the time complexity of each operation and saw why some operations are more efficient than others. Operations at the end of the array are generally fast, while operations at the beginning or middle often require shifting elements and therefore take more time.

Although arrays are one of the simplest data structures, they form the foundation of many more advanced structures and algorithms. Understanding how arrays work internally provides valuable insight into how data is stored, accessed, and manipulated efficiently.

In the next section, we will explore one of the most important applications of arrays: **searching**. Specifically, we will study **Linear Search** and **Binary Search**, two fundamental algorithms used to locate data within an array.

References

https://drive.google.com/file/d/18wXJmAyHVrIoU2XspveSV1hGezBzSWEC/view?usp=drive_link

<https://www.geeksforgeeks.org/python/python-arrays/>

<https://www.educative.io/blog/array-operations-in-python>

Next Section:

 [7.2. Searching in Arrays](#)